

Dynamic Programming – Basic Problems

Dice Throw Problem, Binomial Coefficient, Assembly Line Scheduling

Dr. SRIRAMYA P

P BHANU PRAKASH (192425263)

Dice Throw Problem

1. Write a program to determine the number of ways a player can unlock a reward in an online game. The game uses 3 six-faced dice, and the reward is granted only when the total score obtained is exactly 8. Display the total number of possible combinations that produce the target score.

Input:

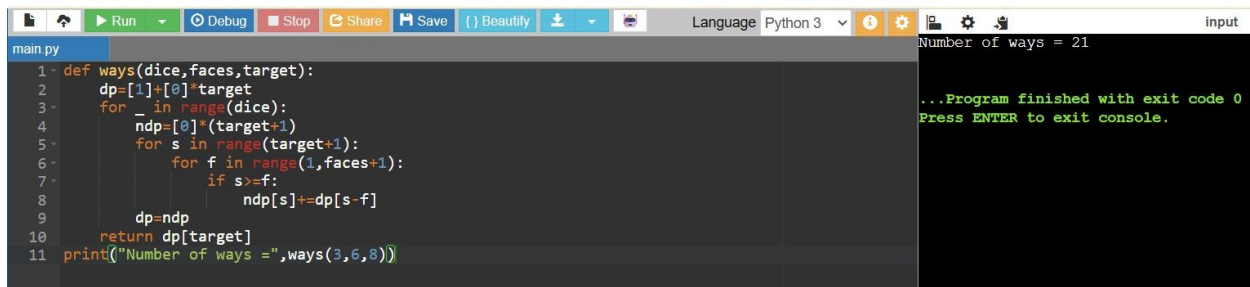
Number of Dice = 3

Faces per Dice = 6

Target Score = 8

Output:

Number of ways = 21



```
main.py
1 def ways(dice, faces, target):
2     dp = [1] + [0] * target
3     for _ in range(dice):
4         ndp = [0] * (target + 1)
5         for s in range(target + 1):
6             for f in range(1, faces + 1):
7                 if s >= f:
8                     ndp[s] += dp[s - f]
9         dp = ndp
10    return dp[target]
11    print("Number of ways =", ways(3, 6, 8))
```

Number of ways = 21

...Program finished with exit code 0
Press ENTER to exit console.

Question 2: Board Game Tournament

Write a program to calculate the number of possible outcomes in a board game tournament where a player rolls 2 dice, each having 6 faces. To qualify for the next round, the player must obtain a total score of 7.

Input:

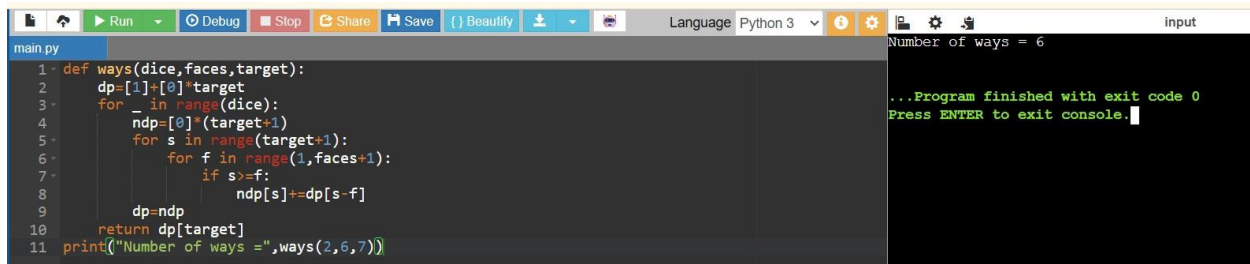
Number of Dice = 2

Faces per Dice = 6

Target Score = 7

Output:

Number of ways = 6



```
1 def ways(dice, faces, target):
2     dp=[1]*[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(2, 6, 7))
```

Number of ways = 6

...Program finished with exit code 0
Press ENTER to exit console.

Question 3: Warehouse Robot Navigation

A warehouse robot moves forward based on the values obtained from 4 dice throws. Each dice has 4 faces. The robot reaches its destination only if the sum of all throws equals 10. Write a program to determine the number of possible movement sequences.

Input:

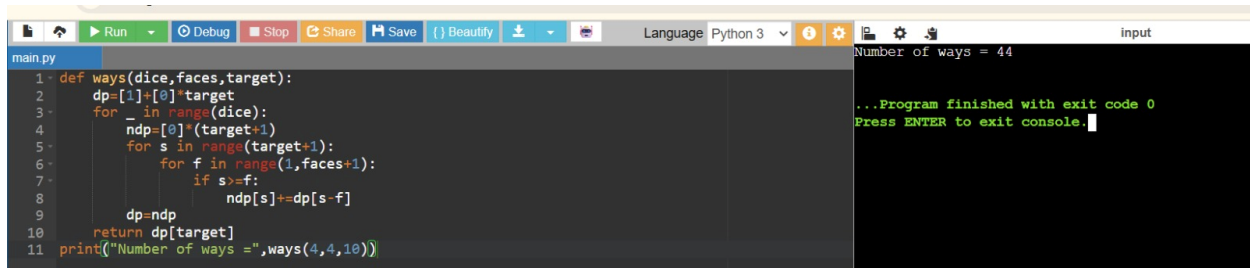
Number of Dice = 4

Faces per Dice = 4

Target Distance = 10

Output:

Number of ways = 44



```
1 def ways(dice, faces, target):
2     dp=[1]*[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(4, 4, 10))
```

Number of ways = 44

...Program finished with exit code 0
Press ENTER to exit console.

Question 4: Casino Reward Calculation

A casino rewards customers who obtain exactly 12 points by rolling 3 six-faced dice. Write a program to calculate the total number of winning combinations.

Input:

Number of Dice = 3

Faces per Dice = 6

Target Sum = 12

Output:

Number of ways = 25

```
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(3, 6, 12))
```

Number of ways = 25

...Program finished with exit code 0
Press ENTER to exit console.

Question 5: Quiz Competition Scoring

In a quiz application, bonus points are awarded if the sum of 5 dice rolls equals 15. Each die has 6 faces. Write a program to find the number of valid scoring combinations.

Input:

Number of Dice = 5

Faces per Dice = 6

Target Score = 15

Output:

Number of ways = 651

```
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(5, 6, 15))
```

Number of ways = 651

...Program finished with exit code 0
Press ENTER to exit console.

Question 6: Sports Prediction System

Write a program to determine the number of ways a predicted score of 9 can be obtained using 3 five-faced dice.

Input:

Number of Dice = 3

Faces per Dice = 5

Target Score = 9

Output:

Number of ways = 19

```
main.py
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(3, 5, 9))
```

input

Number of ways = 19

...Program finished with exit code 0
Press ENTER to exit console.

7. A lottery simulation system uses 2 ten-faced dice. Find the number of possible combinations that result in a sum of 11.

Input:

Number of Dice = 2

Faces per Dice = 10

Target Sum = 11

Output:

Number of ways = 10

```
main.py
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(2, 10, 11))
```

input

Number of ways = 10

...Program finished with exit code 0
Press ENTER to exit console.

8. Write a program to find the number of ways to obtain a given sum using N dice, each having M faces.

Input:

Number of Dice = 2

Faces per Dice = 6

Target Sum = 7

Output:

Number of ways = 6

```
main.py
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(2, 6, 7))
```

input

Number of ways = 6

...Program finished with exit code 0
Press ENTER to exit console.

9. Write a program to determine the number of ways to obtain a target score using multiple dice.

Input:

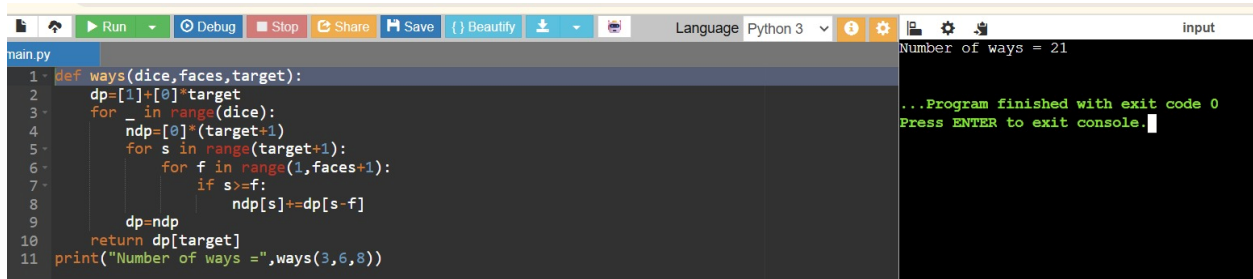
Number of Dice = 3

Faces per Dice = 6

Target Sum = 8

Output:

Number of ways = 21



```
main.py
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(3, 6, 8))
```

Number of ways = 21

...Program finished with exit code 0
Press ENTER to exit console.

10. Write a program to count the possible outcomes that produce a specified sum.

Input:

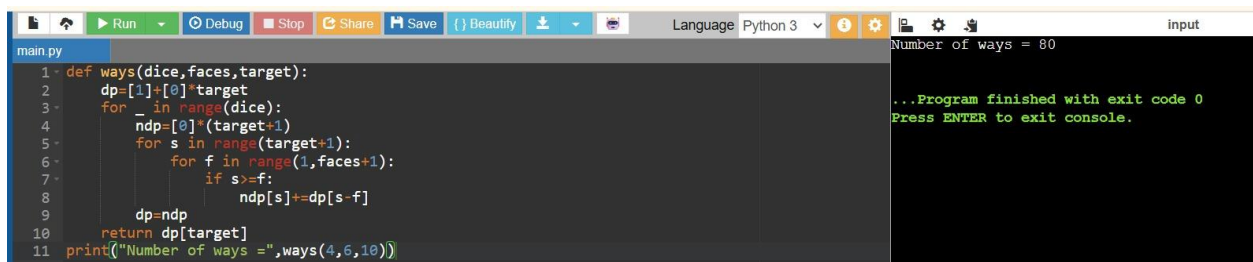
Number of Dice = 4

Faces per Dice = 6

Target Sum = 10

Output:

Number of ways = 80



```
main.py
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(4, 6, 10))
```

Number of ways = 80

...Program finished with exit code 0
Press ENTER to exit console.

11. Write a program to find the number of possible dice combinations that yield the required score.

Input:

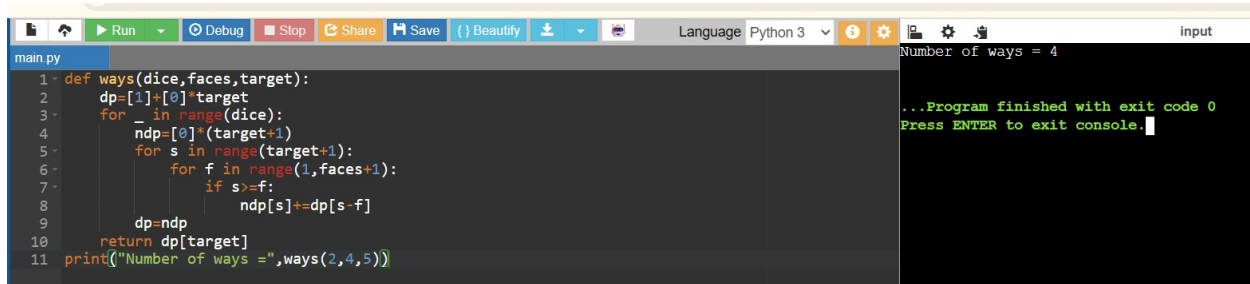
Number of Dice = 2

Faces per Dice = 4

Target Sum = 5

Output:

Number of ways = 4



```
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(2, 4, 5))
```

Number of ways = 4

...Program finished with exit code 0
Press ENTER to exit console.

Binomial Coefficient

1. Write a program to determine the number of ways to obtain the target sum using dice throws.

Input:

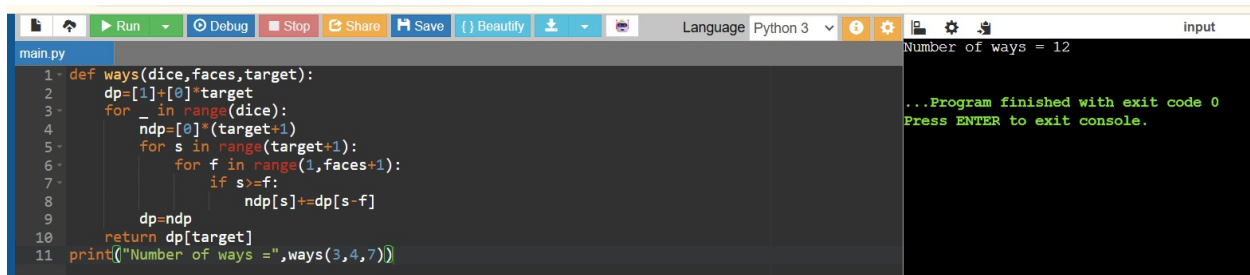
Number of Dice = 3

Faces per Dice = 4

Target Sum = 7

Output:

Number of ways = 12



```
1 def ways(dice, faces, target):
2     dp=[1]+[0]*target
3     for _ in range(dice):
4         ndp=[0]*(target+1)
5         for s in range(target+1):
6             for f in range(1, faces+1):
7                 if s>=f:
8                     ndp[s]+=dp[s-f]
9         dp=ndp
10    return dp[target]
11    print("Number of ways =", ways(3, 4, 7))
```

Number of ways = 12

...Program finished with exit code 0
Press ENTER to exit console.

2. Write a program to calculate the Binomial Coefficient $C(n, r)$.

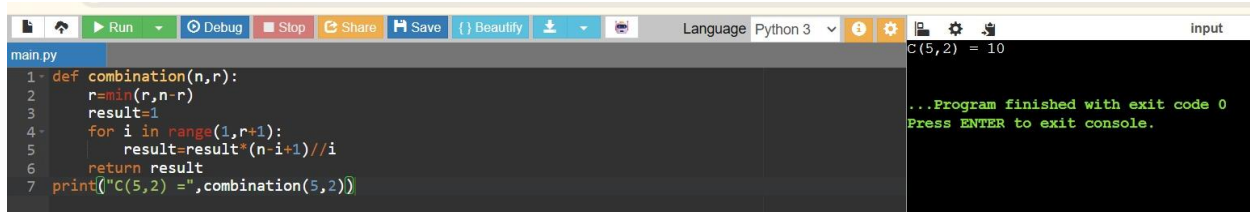
Input:

$n = 5$

$r = 2$

Output:

$C(5, 2) = 10$



```
1 def combination(n, r):
2     r=min(r, n-r)
3     result=1
4     for i in range(1, r+1):
5         result=result*(n-i+1)//i
6     return result
7    print("C(5, 2) =", combination(5, 2))
```

C(5, 2) = 10

...Program finished with exit code 0
Press ENTER to exit console.

3. Write a program to find the number of ways to select r items from n items.

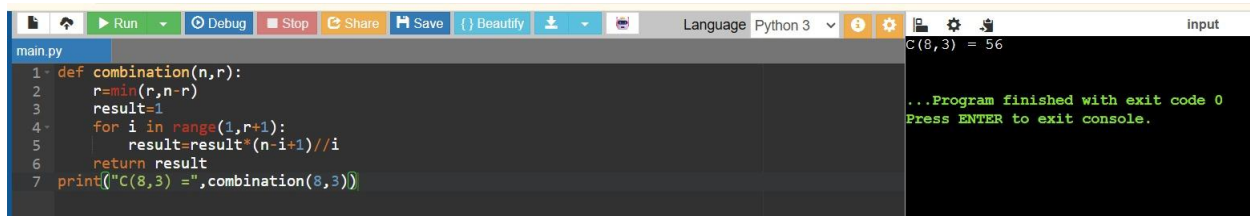
Input:

$n = 8$

$r = 3$

Output:

$C(8,3) = 56$



```
main.py
1 def combination(n,r):
2     r=min(r,n-r)
3     result=1
4     for i in range(1,r+1):
5         result=result*(n-i+1)//i
6     return result
7 print("C(8,3) =",combination(8,3))
```

input
C(8,3) = 56
...Program finished with exit code 0
Press ENTER to exit console.

4. Write a program to compute the Binomial Coefficient using Dynamic Programming.

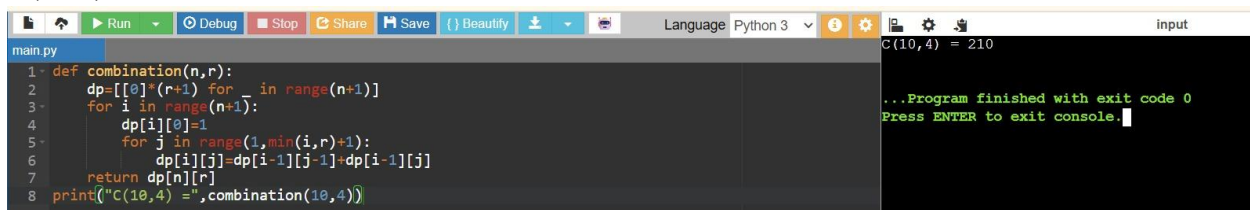
Input:

$n = 10$

$r = 4$

Output:

$C(10,4) = 210$



```
main.py
1 def combination(n,r):
2     dp=[[0]*(r+1) for _ in range(n+1)]
3     for i in range(n+1):
4         dp[i][0]=1
5         for j in range(1,min(i,r)+1):
6             dp[i][j]=dp[i-1][j-1]+dp[i-1][j]
7     return dp[n][r]
8 print("C(10,4) =",combination(10,4))
```

input
C(10,4) = 210
...Program finished with exit code 0
Press ENTER to exit console.

5. Write a program to determine the number of possible committees formed from a group.

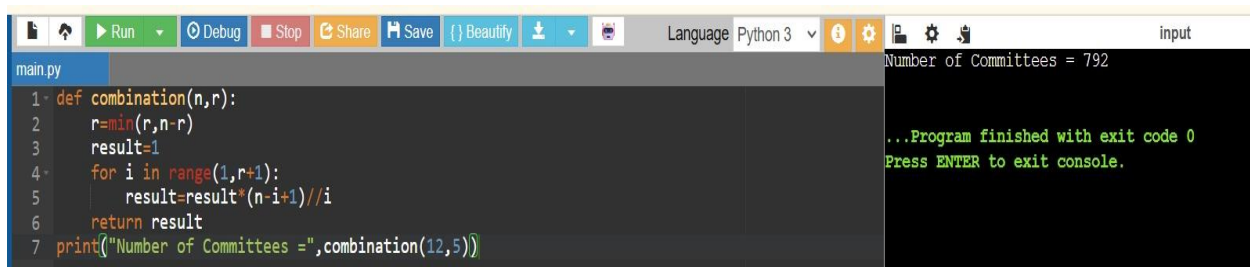
Input:

Total Members = 12

Committee Size = 5

Output:

Number of Committees = 792



```
main.py
1 def combination(n,r):
2     r=min(r,n-r)
3     result=1
4     for i in range(1,r+1):
5         result=result*(n-i+1)//i
6     return result
7 print("Number of Committees =",combination(12,5))
```

input
Number of Committees = 792
...Program finished with exit code 0
Press ENTER to exit console.

6. Write a program to calculate the combination value for given n and r.

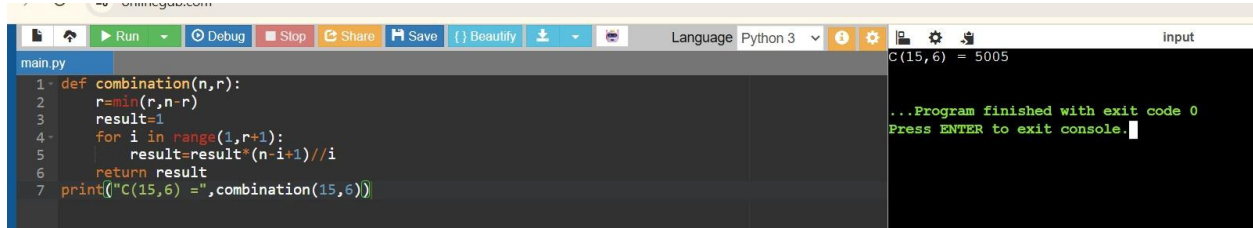
Input:

n = 15

r = 6

Output:

$C(15,6) = 5005$



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'combination(n,r)' that calculates the combination value using a loop. It then prints the result of 'combination(15,6)'. The output in the console is 'C(15,6) = 5005'.

```
1 def combination(n,r):
2     r=min(r,n-r)
3     result=1
4     for i in range(1,r+1):
5         result=result*(n-i+1)//i
6     return result
7 print("C(15,6) =",combination(15,6))
```

Output: C(15,6) = 5005

7. A software company has 10 developers and wants to select 3 developers for a special AI project. Write a program to calculate the number of different teams that can be formed.

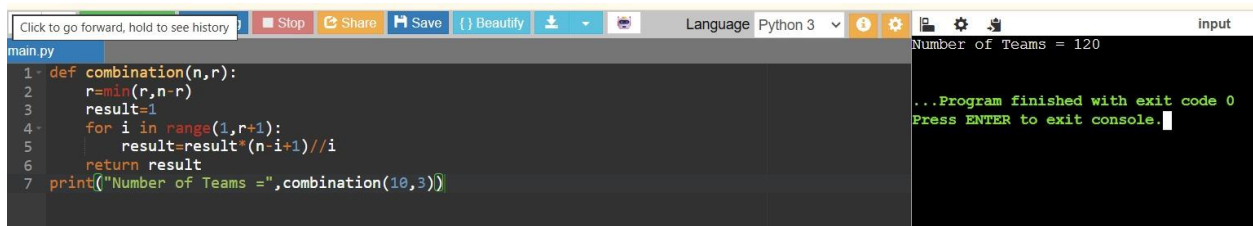
Input:

Total Developers = 10

Developers Required = 3

Output:

Number of Teams = 120



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'combination(n,r)' that calculates the combination value using a loop. It then prints the result of 'combination(10,3)'. The output in the console is 'Number of Teams = 120'.

```
1 def combination(n,r):
2     r=min(r,n-r)
3     result=1
4     for i in range(1,r+1):
5         result=result*(n-i+1)//i
6     return result
7 print("Number of Teams =",combination(10,3))
```

Output: Number of Teams = 120

8. A university must select 4 faculty members from a pool of 12 professors to form a scholarship committee. Write a program to determine the total number of possible committees.

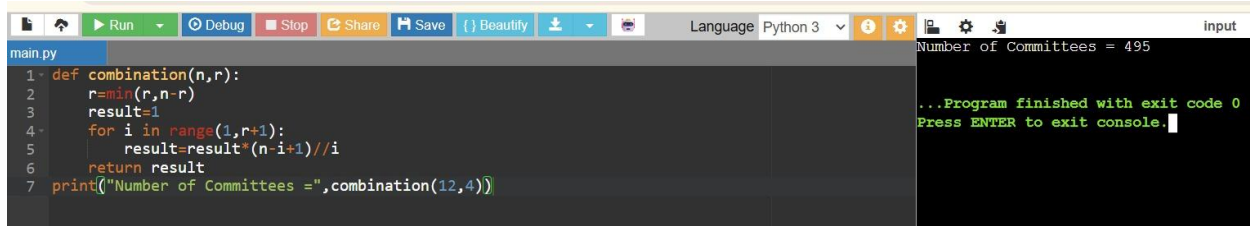
Input:

Total Professors = 12

Committee Size = 4

Output:

Number of Committees = 495



```
main.py
1 def combination(n,r):
2     r=min(r,n-r)
3     result=1
4     for i in range(1,r+1):
5         result=result*(n-i+1)//i
6     return result
7 print("Number of Committees =",combination(12,4))
```

Number of Committees = 495

...Program finished with exit code 0
Press ENTER to exit console.

9. A cricket academy has 15 players and needs to choose 5 players for a tournament squad. Calculate the number of possible selections.

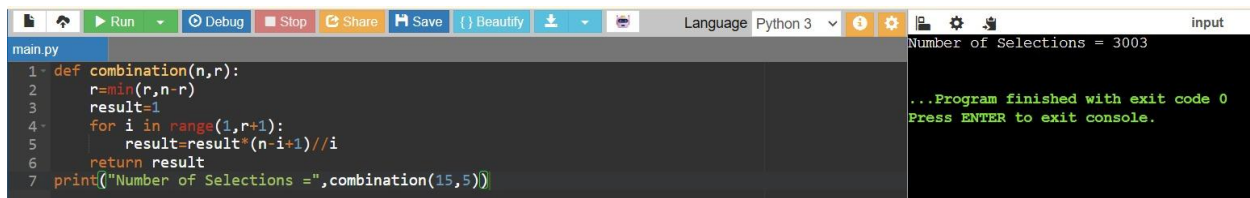
Input:

Total Players = 15

Players Required = 5

Output:

Number of Selections = 3003



```
main.py
1 def combination(n,r):
2     r=min(r,n-r)
3     result=1
4     for i in range(1,r+1):
5         result=result*(n-i+1)//i
6     return result
7 print("Number of Selections =",combination(15,5))
```

Number of Selections = 3003

...Program finished with exit code 0
Press ENTER to exit console.

10. A company wants to select 6 customers from 20 volunteers to test a new product. Write a program to determine the total number of testing groups.

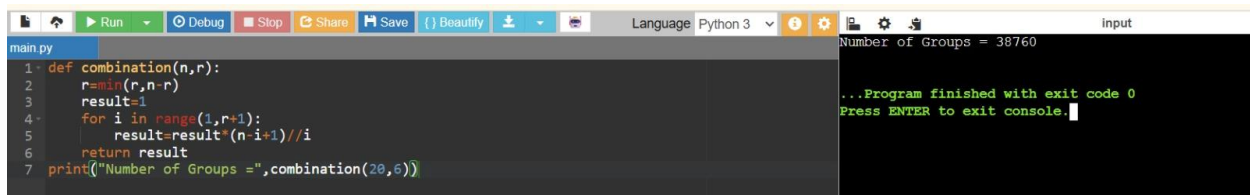
Input:

Total Volunteers = 20

Group Size = 6

Output:

Number of Groups = 38760



```
main.py
1 def combination(n,r):
2     r=min(r,n-r)
3     result=1
4     for i in range(1,r+1):
5         result=result*(n-i+1)//i
6     return result
7 print("Number of Groups =",combination(20,6))
```

Number of Groups = 38760

...Program finished with exit code 0
Press ENTER to exit console.

11. A cloud provider has 8 servers and must allocate 3 servers to a customer. Determine the number of possible server combinations.

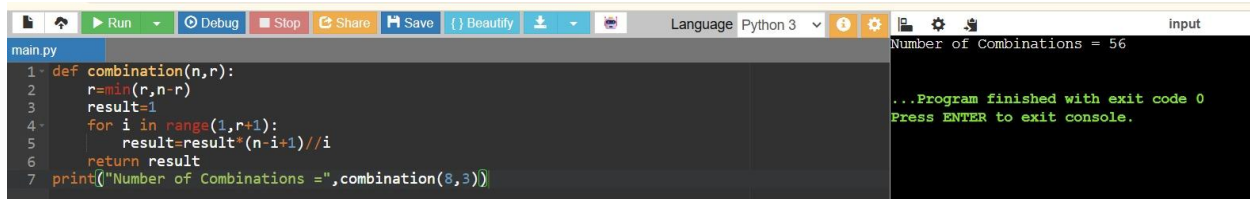
Input:

Total Servers = 8

Servers Selected = 3

Output:

Number of Combinations = 56



```
main.py
1- def combination(n,r):
2-     r=min(r,n-r)
3-     result=1
4-     for i in range(1,r+1):
5-         result=result*(n-i+1)//i
6-     return result
7- print("Number of Combinations =",combination(8,3))
```

Number of Combinations = 56

...Program finished with exit code 0
Press ENTER to exit console.

Assembly Line Scheduling

1. Write a program to find the minimum time required to manufacture a product using two assembly lines.

Input:

Entry Times = [10,12]

Exit Times = [18,7]

Line1 = [4,5,3,2]

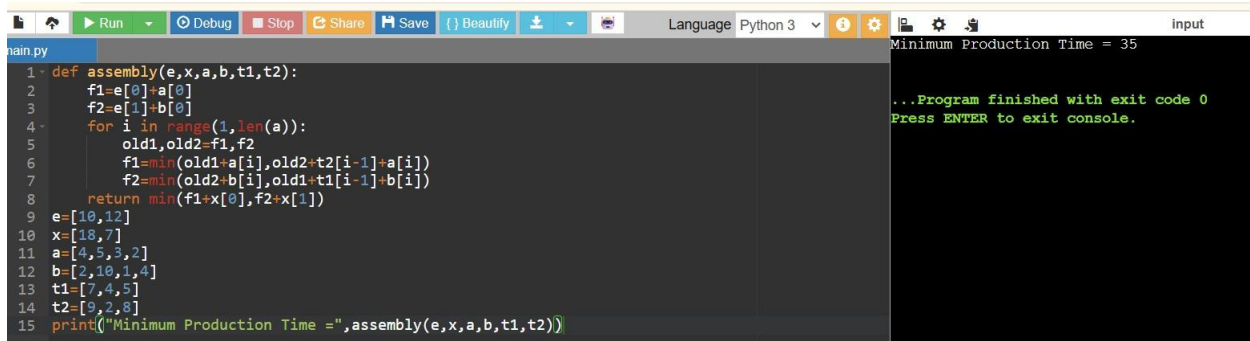
Line2 = [2,10,1,4]

Transfer1 = [7,4,5]

Transfer2 = [9,2,8]

Output:

Minimum Production Time = 35



```
main.py
1- def assembly(e,x,a,b,t1,t2):
2-     f1=e[0]+a[0]
3-     f2=e[1]+b[0]
4-     for i in range(1,len(a)):
5-         old1,old2=f1,f2
6-         f1=min(old1+a[i],old2+t2[i-1]+a[i])
7-         f2=min(old2+b[i],old1+t1[i-1]+b[i])
8-     return min(f1+x[0],f2+x[1])
9- e=[10,12]
10- x=[18,7]
11- a=[4,5,3,2]
12- b=[2,10,1,4]
13- t1=[7,4,5]
14- t2=[9,2,8]
15- print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

Minimum Production Time = 35

...Program finished with exit code 0
Press ENTER to exit console.

2. Write a program to determine the optimal assembly line path with minimum processing time.

Input:

Entry Times = [8,10]

Exit Times = [12,15]

Line1 = [5,6,4]

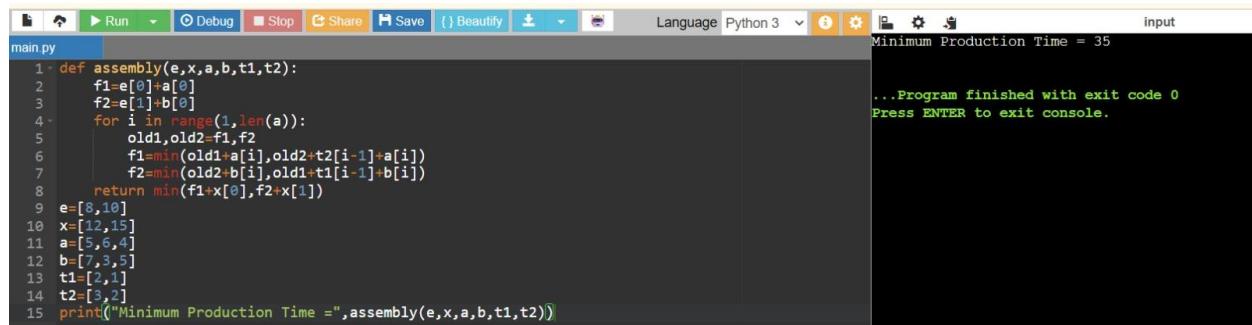
Line2 = [7,3,5]

Transfer1 = [2,1]

Transfer2 = [3,2]

Output:

Minimum Production Time = 33



```
main.py
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1,old2=f1,f2
6         f1=min(old1+a[i],old2+t2[i-1]+a[i])
7         f2=min(old2+b[i],old1+t1[i-1]+b[i])
8     return min(f1+x[0],f2+x[1])
9 e=[8,10]
10 x=[12,15]
11 a=[5,6,4]
12 b=[7,3,5]
13 t1=[2,1]
14 t2=[3,2]
15 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

input

Minimum Production Time = 33

...Program finished with exit code 0
Press ENTER to exit console.

3. Write a program to compute the fastest route through assembly stations.

Input:

Entry Times = [5,6]

Exit Times = [4,5]

Line1 = [3,4,5]

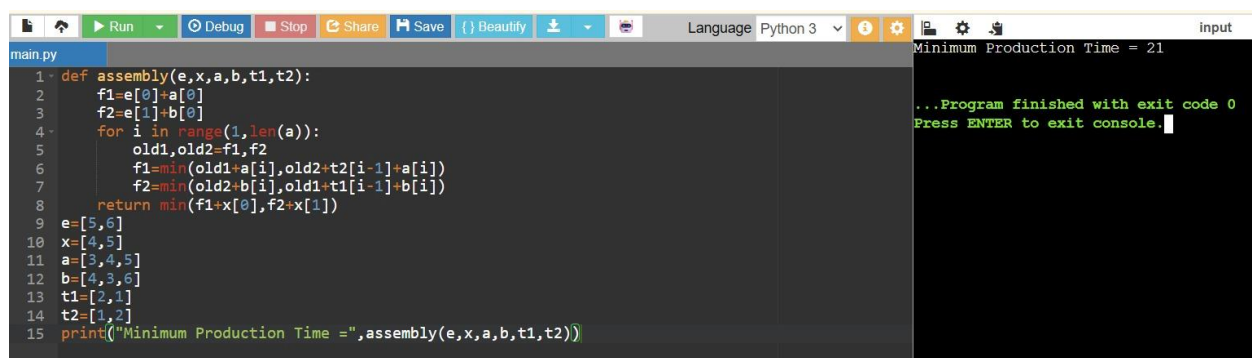
Line2 = [4,3,6]

Transfer1 = [2,1]

Transfer2 = [1,2]

Output:

Minimum Production Time = 21



```
main.py
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1,old2=f1,f2
6         f1=min(old1+a[i],old2+t2[i-1]+a[i])
7         f2=min(old2+b[i],old1+t1[i-1]+b[i])
8     return min(f1+x[0],f2+x[1])
9 e=[5,6]
10 x=[4,5]
11 a=[3,4,5]
12 b=[4,3,6]
13 t1=[2,1]
14 t2=[1,2]
15 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

input

Minimum Production Time = 21

...Program finished with exit code 0
Press ENTER to exit console.

4. Write a program to identify the assembly line schedule that minimizes total manufacturing time.

Input:

Entry Times = [7,9]

Exit Times = [8,6]

Line1 = [4,3,5,2]

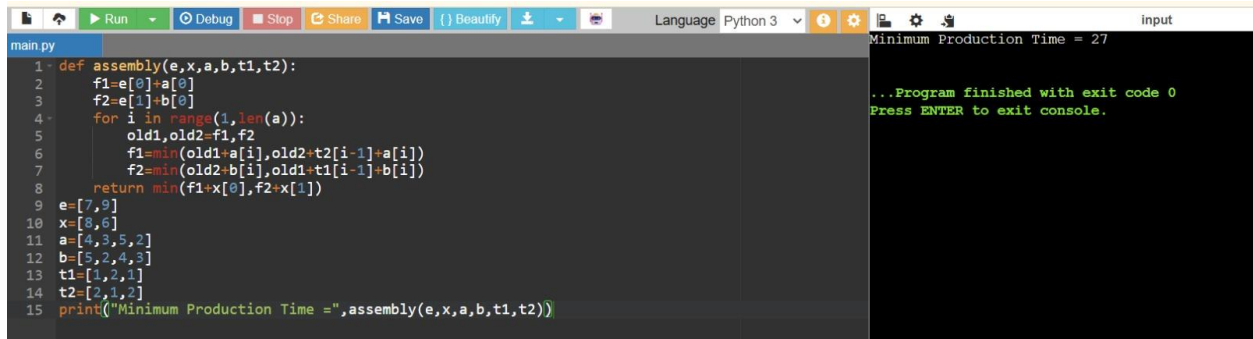
Line2 = [5,2,4,3]

Transfer1 = [1,2,1]

Transfer2 = [2,1,2]

Output:

Minimum Production Time = 29



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'assembly' that takes parameters (e, x, a, b, t1, t2) and calculates the minimum production time. It uses a loop to iterate over the elements of 'a' and 'b', updating 'old1' and 'old2' values. The final result is printed as 'Minimum Production Time = 29'. The console output on the right shows the same result and a message indicating the program finished with exit code 0.

```
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1,old2=f1,f2
6         f1=min(old1+a[i],old2+t2[i-1]+a[i])
7         f2=min(old2+b[i],old1+t1[i-1]+b[i])
8     return min(f1+x[0],f2+x[1])
9 e=[7,9]
10 x=[8,6]
11 a=[4,3,5,2]
12 b=[5,2,4,3]
13 t1=[1,2,1]
14 t2=[2,1,2]
15 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

5. Write a program to determine the minimum cost path through an assembly system.

Input:

Entry Times = [6,7]

Exit Times = [5,4]

Line1 = [2,4,3]

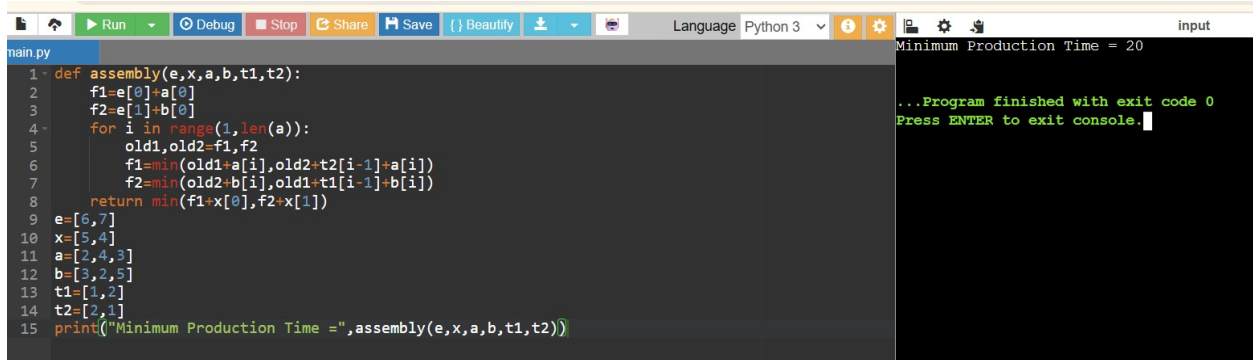
Line2 = [3,2,5]

Transfer1 = [1,2]

Transfer2 = [2,1]

Output:

Minimum Production Time = 18



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'assembly' that takes parameters (e, x, a, b, t1, t2) and calculates the minimum production time. It uses a loop to iterate over the elements of 'a' and 'b', updating 'old1' and 'old2' values. The final result is printed as 'Minimum Production Time = 20'. The console output on the right shows the same result and a message indicating the program finished with exit code 0.

```
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1,old2=f1,f2
6         f1=min(old1+a[i],old2+t2[i-1]+a[i])
7         f2=min(old2+b[i],old1+t1[i-1]+b[i])
8     return min(f1+x[0],f2+x[1])
9 e=[6,7]
10 x=[5,4]
11 a=[2,4,3]
12 b=[3,2,5]
13 t1=[1,2]
14 t2=[2,1]
15 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

6. An automobile manufacturer has two assembly lines for producing cars. Each line contains four stations. The processing times at each station and transfer times between lines are provided. Write a program to determine the minimum time required to manufacture a car.

Input:

Entry Times = [10,12]

Exit Times = [18,7]

Line1 = [4,5,3,2]

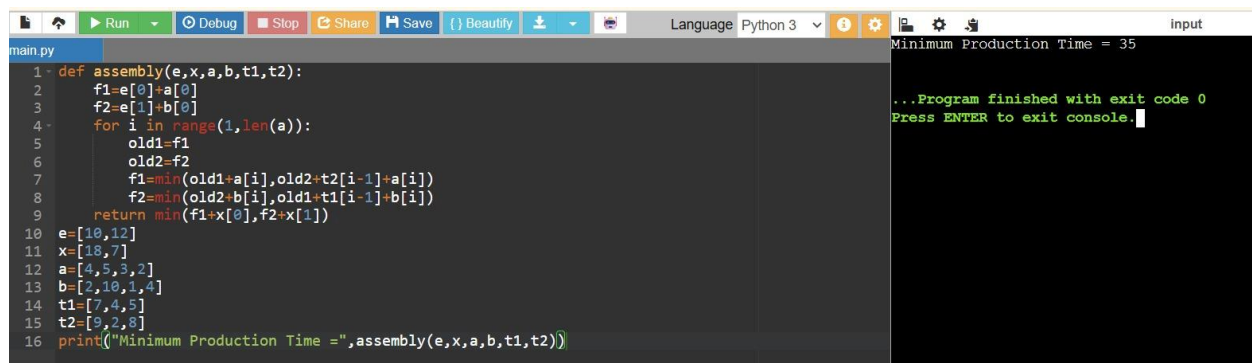
Line2 = [2,10,1,4]

Transfer1 = [7,4,5]

Transfer2 = [9,2,8]

Output:

Minimum Production Time = 35



```
main.py
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1=f1
6         old2=f2
7         f1=min(old1+a[i],old2+t2[i-1]+a[i])
8         f2=min(old2+b[i],old1+t1[i-1]+b[i])
9     return min(f1+x[0],f2+x[1])
10 e=[10,12]
11 x=[18,7]
12 a=[4,5,3,2]
13 b=[2,10,1,4]
14 t1=[7,4,5]
15 t2=[9,2,8]
16 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

Minimum Production Time = 35

...Program finished with exit code 0
Press ENTER to exit console.

7. A smartphone company assembles devices using two production lines. Components may switch lines after completing any station. Determine the optimal route that minimizes total production time.

Input:

Entry Times = [8,10]

Exit Times = [12,15]

Line1 = [5,6,4]

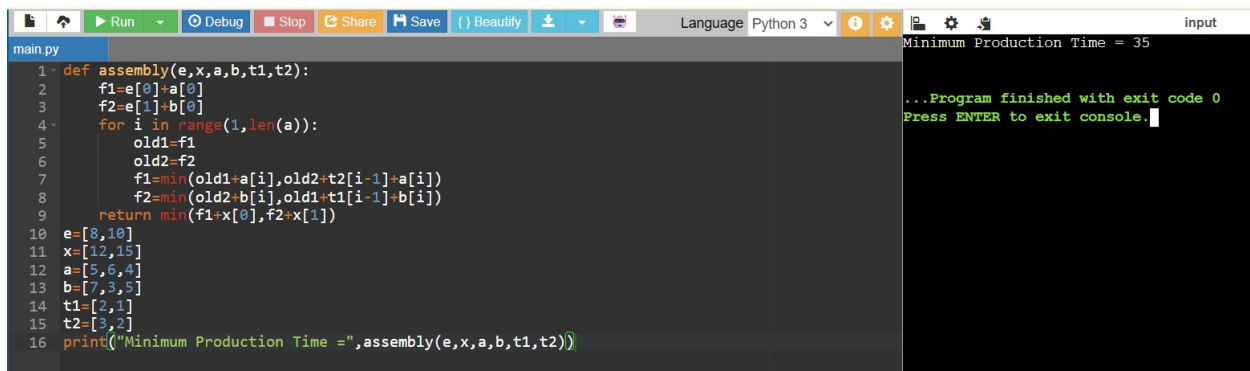
Line2 = [7,3,5]

Transfer1 = [2,1]

Transfer2 = [3,2]

Output:

Minimum Production Time = 33



```
main.py
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1=f1
6         old2=f2
7         f1=min(old1+a[i],old2+t2[i-1]+a[i])
8         f2=min(old2+b[i],old1+t1[i-1]+b[i])
9     return min(f1+x[0],f2+x[1])
10 e=[8,10]
11 x=[12,15]
12 a=[5,6,4]
13 b=[7,3,5]
14 t1=[2,1]
15 t2=[3,2]
16 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

Minimum Production Time = 35

...Program finished with exit code 0
Press ENTER to exit console.

8.A food processing company uses two packaging lines. Products pass through labeling, sealing, and packing stations. Write a program to find the minimum completion time.

Input:

Entry Times = [5,6]

Exit Times = [4,5]

Line1 = [3,4,5]

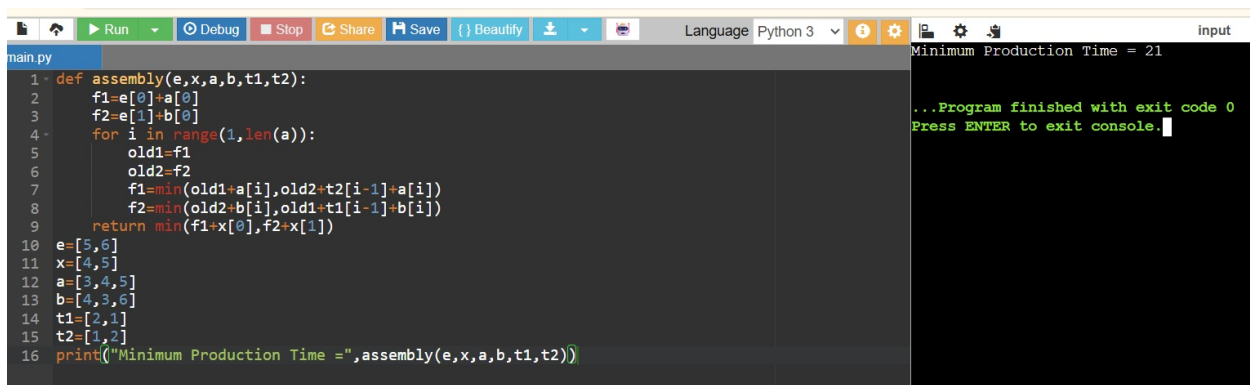
Line2 = [4,3,6]

Transfer1 = [2,1]

Transfer2 = [1,2]

Output:

Minimum Production Time = 21



```
main.py
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1=f1
6         old2=f2
7         f1=min(old1+a[i],old2+t2[i-1]+a[i])
8         f2=min(old2+b[i],old1+t1[i-1]+b[i])
9     return min(f1+x[0],f2+x[1])
10 e=[5,6]
11 x=[4,5]
12 a=[3,4,5]
13 b=[4,3,6]
14 t1=[2,1]
15 t2=[1,2]
16 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

Minimum Production Time = 21

...Program finished with exit code 0
Press ENTER to exit console.

9.Medicine bottles pass through filling, capping, labeling, and packaging stations on two assembly lines. Determine the optimal production schedule.

Input:

Entry Times = [7,9]

Exit Times = [8,6]

Line1 = [4,3,5,2]

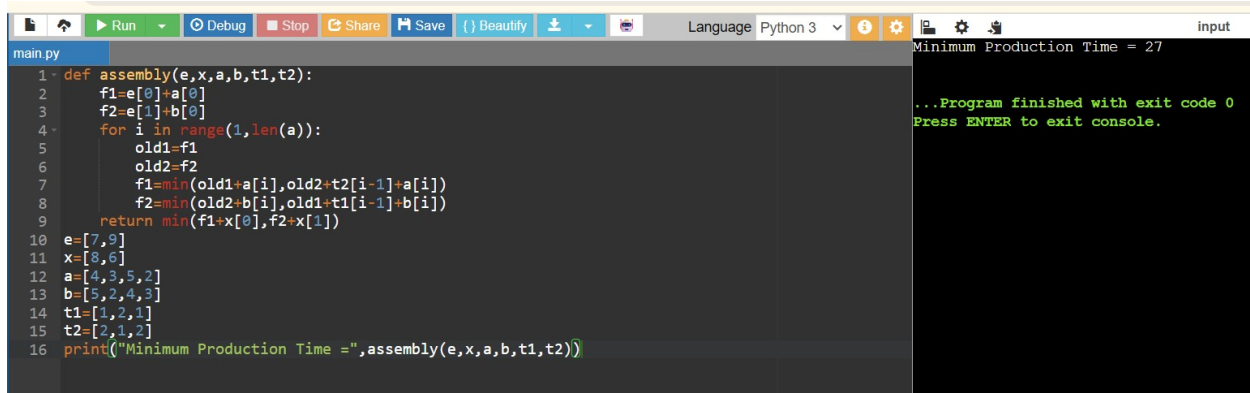
Line2 = [5,2,4,3]

Transfer1 = [1,2,1]

Transfer2 = [2,1,2]

Output:

Minimum Production Time = 27



```
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1=f1
6         old2=f2
7         f1=min(old1+a[i],old2+t2[i-1]+a[i])
8         f2=min(old2+b[i],old1+t1[i-1]+b[i])
9     return min(f1+x[0],f2+x[1])
10 e=[7,9]
11 x=[8,6]
12 a=[4,3,5,2]
13 b=[5,2,4,3]
14 t1=[1,2,1]
15 t2=[2,1,2]
16 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

Minimum Production Time = 27

...Program finished with exit code 0
Press ENTER to exit console.

10.A circuit board manufacturing facility uses two parallel assembly lines. Products can switch lines between stations. Find the minimum manufacturing time.

Input:

Entry Times = [6,7]

Exit Times = [5,4]

Line1 = [2,4,3]

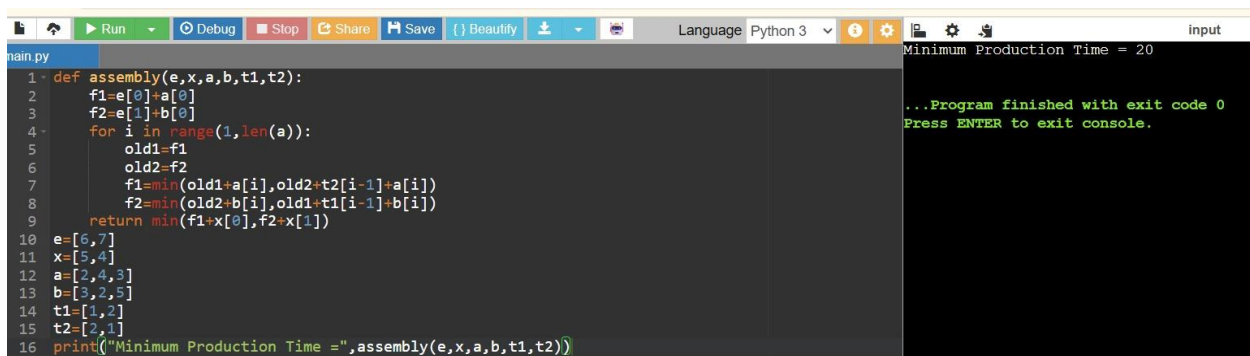
Line2 = [3,2,5]

Transfer1 = [1,2]

Transfer2 = [2,1]

Output:

Minimum Production Time = 20



```
1 def assembly(e,x,a,b,t1,t2):
2     f1=e[0]+a[0]
3     f2=e[1]+b[0]
4     for i in range(1,len(a)):
5         old1=f1
6         old2=f2
7         f1=min(old1+a[i],old2+t2[i-1]+a[i])
8         f2=min(old2+b[i],old1+t1[i-1]+b[i])
9     return min(f1+x[0],f2+x[1])
10 e=[6,7]
11 x=[5,4]
12 a=[2,4,3]
13 b=[3,2,5]
14 t1=[1,2]
15 t2=[2,1]
16 print("Minimum Production Time =",assembly(e,x,a,b,t1,t2))
```

Minimum Production Time = 20

...Program finished with exit code 0
Press ENTER to exit console.